ARTIFICIAL INTELLIGENCE CSA1701 LAB OBSERVATON
Mohammed Fareeth S A (192511288)

EXPERIMENT 12:

Aim

To implement and solve the classical Tic-Tac-Toe Game using the Minimax Algorithm in Python to enable an AI agent to play optimally against a human opponent (or simulate optimal play).

Algorithm

1. Define Board and Players: Represent the 3x3 board using a list or matrix initialized with empty spaces. Define two players: Maximizing Player (AI, e.g., 'X') and Minimizing Player (Human, e.g., 'O').
2. Evaluation Function: Evaluate the board state:
    - Returns $+10$ if the AI wins.
    - Returns -10 if the opponent wins.
    - Returns 0 if it's a tie or the game is ongoing.
3. Minimax Recursive Function:
    - Check if the terminal state (win, loss, or tie) has been reached; if so, return the score.
    - If it is the maximizer's turn, loop through all available moves, simulate the move, call Minimax recursively, and find the maximum score.
    - If it is the minimizer's turn, loop through all available moves, simulate the move, call Minimax recursively, and find the minimum score.
4. Best Move Selection: The AI evaluates all legal successor moves using the Minimax function and selects the move that yields the highest score.

Code:

```python
class TicTacToe:
    def __init__(self):
        self.board = [[' ' for _ in range(3)] for _ in range(3)]
        self.ai = 'X'
        self.human = 'O'

    def print_board(self):
        for row in self.board:
            print("|" + "|".join(row) + "|")
        print()

    def is_moves_left(self):
        return any(cell == ' ' for row in self.board for cell in row)

    def evaluate(self):
        for row in self.board:
            if row[0] == row[1] == row[2] and row[0] != ' ':
                return 10 if row[0] == self.ai else -10

        for col in range(3):
            if self.board[0][col] == self.board[1][col] == self.board[2][col] and self.board[0][col] != ' ':
                return 10 if self.board[0][col] == self.ai else -10

        if self.board[0][0] == self.board[1][1] == self.board[2][2] and self.board[0][0] != ' ':
            return 10 if self.board[0][0] == self.ai else -10

        if self.board[0][2] == self.board[1][1] == self.board[2][0] and self.board[0][2] != ' ':
            return 10 if self.board[0][2] == self.ai else -10

        return 0

    def minimax(self, depth, is_max):
        score = self.evaluate()

        if score == 10:
            return score
        if score == -10:
            return score
        if not self.is_moves_left():
            return 0

        if is_max:
            best = -float('inf')
            for i in range(3):
                for j in range(3):
                    if self.board[i][j] == ' ':
                        self.board[i][j] = self.ai
                        best = max(best, self.minimax(depth + 1, not is_max))
                        self.board[i][j] = ' '
            return best
        else:
            best = float('inf')
```

```
Current Board State:
|X|O|X|
|O|O| |
| | |X|

Optimal AI Move for 'X': Row 1, Column 2
```

Result

The Minimax algorithm successfully evaluated all possible future game states from the given board configuration. By alternating between maximizing and minimizing outcomes, the AI identified the optimal coordinate placement to secure a winning advantage or block the opponent.

Conclusion

The program successfully demonstrated the implementation of adversarial search using the Minimax algorithm for Tic-Tac-Toe. By exhaustively exploring the game tree, the AI agent is guaranteed never to lose, making optimal decisions at every turn.